HACK CACHE

Back

PROBLEM STATEMENT

Smart Queue Management and Accessible Navigation for Government Hospitals

Reimagining Government.

Help patients at a government hospital register in their own language, get an OPD slip, find the right ward and know when their turn is coming.

Problem Statement 2: Smart Queue Management and Accessible Navigation for Government Hospitals

The Challenge

Patients visiting government hospitals often spend hours waiting to register, obtain an OPD slip, and consult a doctor. Even after receiving their slip, many struggle to find the correct ward or department—for example, knowing they need to visit Ward 4 but having no idea where it is located.

The registration process itself can be difficult for people unfamiliar with digital services, those who speak regional languages, elderly patients, and people with disabilities. Combined with long queues, unclear signage, overcrowded corridors, and limited information about waiting times, this creates unnecessary stress and delays. Patients requiring urgent medical attention may also face delays if emergencies are not identified promptly.

Your challenge is to build a smart hospital queue management and navigation solution that makes registration easy for everyone, helps patients find their destinations, and reduces unnecessary waiting and confusion throughout their hospital visit.

Possible Areas to Explore

- **Native Language Support:** Support local and regional languages so patients can register and navigate the system in a language they understand.
- **Accessibility and Inclusion:** Design for elderly patients, people with disabilities, users with low digital literacy, and people unfamiliar with smartphones. Consider voice guidance, text-to-speech, readable interfaces, and assisted registration.
- **Smart Queue Management:** Provide token numbers, estimated waiting times, live queue status, and notifications when a patient's turn approaches.
- **Hospital Navigation:** Help patients locate wards, departments, laboratories, pharmacies, and consultation rooms through maps, floor plans, landmarks, or step-by-step directions.
- **Slip-to-Destination Guidance:** Allow patients to enter or scan their OPD slip details and receive clear directions to the assigned ward or department.
- **Emergency Prioritization:** Enable urgent cases to be flagged for immediate assessment by qualified medical staff.
- **Real-Time Updates:** Inform patients about doctor availability, delays, queue changes, and their next steps.
- **Hospital Staff Dashboard:** Help staff manage patient flow, monitor overcrowding, and coordinate between registration counters and departments.

Expected Outcome

Build a working prototype that demonstrates how a patient can easily register in a preferred language, obtain an OPD slip, find the correct department, track their turn, and reach the appropriate doctor with minimal confusion.

Important

You do not need to build a complete hospital management system. You may focus on one high-impact problem or combine multiple features. You can use a sample hospital layout and simulated patient data; integration with an actual hospital system is not mandatory.

Emergency cases must be assessed by qualified medical staff. Digital queues must never delay emergency care, and accessibility should be considered a core requirement rather than an optional add-on.

Evaluation Focus

world usefulness.

BUILD IT WITH AI

Replace anything in [square brackets]. Start with Build 1, and run Build 2 if something is missing. If something breaks, use the Debug prompt that matches. Use only made-up patient data.

BUILD 1 · THE WHOLE PROTOTYPE

Copy

Build a working web app prototype for a hackathon. Build the whole thing, not a plan: every screen below should work when I open it.

OUR CHOICES (change these, or leave them as they are)
- App name: OPD Saathi
- Hospital: City District Hospital, a sample government hospital with three floors
- Main user: Kamla, 62, visiting alone, reads a little Hindi, has a basic Android phone, walks slowly and avoids stairs
- Part of the visit we go deepest on: getting from the OPD slip to the right room

THE PROBLEM
Patients at government hospitals wait hours to register, get an OPD slip and see a doctor, and then can't find the room or ward written on the slip. Registering is hard for elderly patients, people with disabilities, people who don't read English and people who rarely use smartphones. The app lets a patient register in their own language, get an OPD slip with a token, see their place in the queue, find their way, and know when to go in. Staff get a dashboard to run the queues. It does not connect to any real hospital system: the hospital and every patient are made up.

EMERGENCIES COME FIRST (this is not optional)
- An "Emergency" button is on every screen, including before registration. It shows the way to the Emergency (Casualty) department on the map, tells the person to go there now or tell any staff member, and shows 112 (and 108 for an ambulance).
- The first registration screen lists the signs that need care right away (chest pain, trouble breathing, heavy bleeding, fainting or not waking up, sudden weakness or a drooping face, a serious injury, a pregnant woman who is bleeding or in severe pain, a child who is very unwell), with "Any of these? Go to Emergency now. You don't need a token."
- An "I need urgent help" button lets a patient or helper raise a flag at any time. The flag goes to the top of the staff dashboard, with a sound, until a staff member marks it as seen.
- The app never decides whether something is an emergency, never tells anyone they can wait, and never makes an urgent case wait for registration or a token. Only qualified staff assess urgent cases.

HOW TO BUILD IT
- React + Vite + Tailwind CSS, plain JavaScript (no TypeScript), React Router. No backend and no login.

the waiting-room board update live when they are open in different tabs of the same browser.
- Keep the hospital, departments and sample patients in files under src/data: hospital.js (floors, places, the paths between them), departments.js and patients.js. Keep all interface text in src/i18n/en.js and src/i18n/hi.js.
- Draw floor plans and icons as inline SVG. No image downloads.
- Use only made-up data. Never ask for a real Aadhaar or ABHA number. The slip shows the token, first name and department only.
- If you can't create a project (for example, you're a chat assistant), build it in plain HTML, CSS and JavaScript as a few files with no build step, each in its own code block with its file name above it. If you run out of space, stop at the end of a file and I'll say "continue".

THE SAMPLE HOSPITAL (put this in src/data/hospital.js)
- Ground floor: Main entrance, Registration counters, Help desk, Emergency (Casualty), Pharmacy, Laboratory (blood tests), X-ray, Drinking water, Toilets, Wheelchair point, Lift, Stairs, Ramp.
- First floor: OPD rooms 1 to 8 (General Medicine in rooms 1 and 2, Children 3, Women and pregnancy 4, Bones and joints 5, Ear, nose and throat 6, Eyes 7, Skin 8), Waiting hall A, Lift, Stairs, Toilets.
- Second floor: Ward 4 (Medicine), Ward 5 (Surgery), Ward 6 (Children), Nurses' station, Lift, Stairs.
- Each place has a name, a floor, an x/y position on its floor plan, and one landmark people can see ("opposite the blue chairs", "next to the drinking water").
- The paths between places form a graph, with the walking distance in metres and whether the path uses stairs.
- In departments.js, give each department its room, doctor's name, OPD hours (9:00 to 14:00) and a sample average consultation time.

PATIENT SCREENS
1. Home: big buttons with an icon and a word each: Register / I have my slip / Find a place / Emergency. On the first visit, ask for a language, showing each language's name in its own script.
2. Register: one question per screen, with a progress bar ("Step 2 of 5"): name, age, gender, mobile number (optional, for alerts), and "Which doctor do you need?" as picture cards in plain words (General Medicine "fever, cough, BP, sugar"; Children; Women and pregnancy; Bones and joints; Ear, nose and throat; Eyes; Skin). Add "Not sure? Ask at the help desk", which shows the way there. A big microphone button on each text question lets the patient speak instead of typing, where the browser supports it. A "Someone is helping me" switch turns on assisted mode: bigger text, everything read aloud, and a helper can fill it in for them. Back keeps every answer.
3. OPD slip: the token (for example MED-042), department, room and floor, the doctor's name, how many people are ahead, the estimated wait, and a QR code that opens this patient's queue screen. It can be saved as an image, printed, read aloud, and shown in large type to staff.
4. My turn: the patient's place in the queue ("6 people ahead of you, about 25 minutes"), the token being served now, the doctor's status (in the room / on a break until 11:30 / running late), and what to do next. When 3 people are ahead, the phone vibrates, plays a sound, and shows a banner and a browser notification, and a simulated SMS appears ("Your turn is close. Please go to Room 3 on the first floor"). When the token is called, the screen says so in big type, with directions to the room.
5. I have my slip: enter the token number, or scan the QR code with the camera (use the browser's BarcodeDetector where it exists, and hide the scan button where it doesn't). This opens My turn and

plan in SVG with the route drawn on it, plus step-by-step directions with landmarks ("Take the lift to the first floor. Turn right. Room 3 is the third door on your left, opposite the blue chairs"). Work out the route with Dijkstra's shortest path over the graph in hospital.js. An "Avoid stairs" switch (on by default in assisted mode) routes through lifts and ramps only. A "Listen" button reads each step aloud. An "I'm lost" button shows the way to the nearest help desk, plus a large card to show any staff member with the destination in the patient's language and in English.
7. Emergency: as described under EMERGENCIES COME FIRST.
8. Settings: language (English / हिंदी), text size (three steps), high contrast, read aloud, avoid stairs.

STAFF SCREENS
9. Staff dashboard (/staff): every department's queue, with the token being served now, the waiting count and the average wait. Buttons to call next, mark a no-show, send a patient to the end of the queue, and move a patient to another department. The doctor's status for each room (available / on a break until a time / running late by some minutes), which patients see straight away. An overcrowding colour for each waiting area, based on how many people are waiting compared with its seats. Urgent flags pinned at the top, with a sound, until marked as seen. Say on the screen that a real hospital would need a staff login.
10. Registration counter: register a patient for them in a few taps (for someone with no phone) and print their slip.
11. Waiting-room board (/board): a large-screen view for a TV, showing the token being served in each room, the next three tokens, and any delays, alternating between English and Hindi. Announce each called token aloud ("Token MED-042, please go to Room 1").
12. Demo controls on the staff dashboard: add 20 sample patients, speed time up 10×, and reset everything, so the whole flow can be shown in two minutes.

HOW WAITING TIME WORKS
Estimated wait = the number of people ahead x the average consultation time of the last 10 patients in that department (starting from the sample average in departments.js), plus any delay the doctor is running. Round it to 5 minutes, show it as "about", and update it every time the queue moves.

DESIGN RULES
- Mobile first: designed for a 360px-wide screen, and fine on a laptop and a TV.
- Plain words a Class 8 student would understand. One main action per screen. Every term like OPD, token or Casualty gets a one-line explanation the first time it appears.
- Usable by someone who barely reads: every button has an icon and a word, and every screen can be read aloud.
- Body text at least 18px, buttons at least 56px tall, strong colour contrast (WCAG AA), visible keyboard focus, and a label on every icon and form field for screen readers. Never rely on colour alone.
- Switching language changes every piece of text, including directions, landmarks and announcements.
- Calm and reassuring tone.
- Fast on a weak connection: no big images and no heavy libraries.

- With the patient screen, the staff dashboard and the board open in three tabs, a patient can register, get a slip, show up on the dashboard and the board, be called, and see the call on their own screen.
- An urgent flag raised on the patient screen appears at the top of the staff dashboard within a second.
- Directions are correct between any two places, including across floors, and "Avoid stairs" never uses stairs.
- Every piece of text switches between English and Hindi.
- Everything works at 360px wide with text at the largest size, and survives a page reload.
Then tell me how to run it, and list anything you did not finish.

BUILD 2 · FINISH WHAT'S MISSING Copy

Go through the app you built and check it against this list. For each item, say whether it is done, partly done or missing. Then build everything that is partly done or missing, one item at a time, and keep everything that already works exactly as it is.

- Emergency button on every screen, with the way to Emergency, 112 and 108; the emergency signs list on the first registration screen; urgent flags pinned to the top of the staff dashboard with a sound until marked as seen; nothing ever makes an urgent case wait
- Home with four big buttons and a first-visit language choice
- Register: one question per screen, progress bar, picture cards for the department, "Not sure?" to the help desk, microphone input where supported, assisted mode, Back keeps answers
- OPD slip with token, room, floor, doctor, people ahead, estimated wait and a QR code; it can be saved, printed, read aloud and shown in large type
- My turn: place in the queue, now serving, doctor's status, an alert at 3 ahead (vibration, sound, banner, notification, simulated SMS), and the called screen with directions
- I have my slip: token entry, plus QR scanning where the browser supports it
- Find a place: SVG floor plans with the route drawn, step-by-step directions with landmarks, Dijkstra routing, Avoid stairs, Listen, I'm lost
- Settings: language, text size, high contrast, read aloud, avoid stairs
- Staff dashboard: queues, call next, no-show, end of queue, move department, doctor status, overcrowding colours
- Registration counter, waiting-room board with spoken announcements, demo controls
- Estimated wait from the last 10 consultations plus the doctor's delay, rounded to 5 minutes
- Live sync between tabs with BroadcastChannel and localStorage
- All hospital data in src/data, and all interface text in the translation files
- Works at 360px wide, at the largest text size, after a reload, with no console errors

When you're done, go through every screen once more so spacing, buttons and wording are consistent, and list anything still not finished.

ADD-ON · LIVE ACROSS PHONES Copy

the shared state (queues, tokens, doctor status, urgent flags) to the server, and keep the hospital map and departments as they are.
- Every change (a new registration, call next, a doctor's break, an urgent flag) reaches every open screen within a second.
- If a phone loses its connection, keep showing the last known position with "Last updated 2 minutes ago", reconnect automatically, and catch up when the connection is back.
- Patients can see only their own token and the public queue. Only the staff screens can call, move or remove patients. Add a simple staff PIN for the prototype, and say that a real hospital needs a proper staff login.
- Tell me how to run the server and the app together, and how to put both online so the judges can open the patient screen on their own phones.

ADD-ON · WORKS ON A WEAK CONNECTION Copy

Make the patient screens work on a weak or patchy hospital connection.
- Turn the app into a PWA (with Vite, use vite-plugin-pwa), with an "Install app" button in Settings that appears only when the browser offers it.
- Cache the app, the floor plans, the directions and the patient's own slip, so all of them open with no connection.
- While offline, show one line saying the queue position may be out of date, with the time it last updated, and refresh as soon as the connection is back.
- If the app has a server, keep any registration made offline on the phone and send it when the connection returns, and mark the slip as waiting to be confirmed until then.
- Then tell me how to test it in Chrome (DevTools, Network, Offline) and with a Lighthouse report.

ADD-ON · ANOTHER LANGUAGE Copy

Add [Tamil] to the app as a third language.
- Translate every string in src/i18n, the department cards, every place name and landmark, the directions, the emergency signs list and the board's announcements. Use simple everyday words, and keep terms people see on hospital signs (OPD, Casualty, X-ray) as they are, in brackets.
- Show each language's name in its own script in the language picker.
- Make sure no screen still shows English when the new language is picked, and that longer words don't break any layout at 360px wide or on the board.
- Make Listen and the board's announcements use a voice for that language if the browser has one. If it doesn't, hide Listen for that language and keep the announcements in Hindi and English.

SUBMIT · README, LINK AND DEMO Copy

Get this ready to submit to the hackathon.
1. Write a README.md: the problem in two lines, who the app is for, what it does (one line per screen), how emergencies are handled and why the app never decides who is urgent, the design decisions we made and why (assisted mode, picture cards, avoid stairs, languages, read aloud), how waiting

2. Tell me step by step how to put it online on Vercel or Netlify from a GitHub repo, so the judges can open a link. Include the setting that stops a page refresh from showing a 404.
3. Write a 3-minute demo script for judges who score ease of registration, language accessibility, inclusive design, queue-management efficiency, navigation accuracy, emergency handling, technical execution and real-world usefulness. Show the patient screen, the staff dashboard and the board side by side; register in Hindi in assisted mode; follow the avoid-stairs route to the room; get the "3 ahead" alert; and raise an urgent flag. Include one sentence for each part on the problem it solves.
4. List anything that could go wrong during a live demo, and how to avoid it.

DEBUG · AN ERROR MESSAGE Copy

I'm getting this error:
[paste the full error message from the terminal or the browser console]

It happens when: [what you did, e.g. pressed Call next, ran npm run dev]
The file it points to: [paste the file, or the part the error names]

Explain in two lines what is causing it. Then fix only that: give me the corrected code for each file that changes, with its file name, and don't change anything unrelated. Tell me how to check it's fixed.

DEBUG · BLANK WHITE SCREEN Copy

The app shows a blank white screen.
The browser console (right-click the page, Inspect, Console) shows: [paste any red text, or write "nothing"]
The terminal shows: [paste it]
What I changed last: [describe it]

Find the cause step by step: the console error first, then the entry files (main.jsx and App.jsx), the routes, any import that might be wrong, and saved data in localStorage that no longer matches what the code expects. Fix the cause, not the symptom. Also add a simple error screen, so a crash in one screen shows a message instead of a blank page.

DEBUG · A FEATURE ISN'T WORKING RIGHT Copy

[The screen or feature, e.g. the waiting time] isn't working properly.
What I expected: [e.g. the wait goes up by 15 minutes when the doctor goes on a 15-minute break]
What happens instead: [e.g. it stays the same]
Steps to make it happen: [1, 2, 3]

Read the code for this feature and work out why it behaves this way before changing anything.

HACK CACHE

Back

DEBUG · THE QUEUE DOESN'T UPDATE ON THE OTHER SCREEN Copy

The queue doesn't update on [the other tab / the board / another phone]. What happens: [e.g. I press Call next on the staff dashboard and the patient's screen still shows 6 ahead]

Check these one by one: every change is saved and a BroadcastChannel message is sent; every screen listens for it and re-reads the state; each screen reads fresh state when it opens; two tabs aren't overwriting each other's changes (only change the part that changed, or have one writer); and, if we have a server, that the socket connects, the event names match on both sides, and the client reconnects. While we debug, add a small "updated at" time on each screen. Fix the cause and tell me how to test it with three tabs open.

DEBUG · THE DIRECTIONS ARE WRONG Copy

The directions from [place] to [place] are wrong. The app says: [what it says]. The right way is: [what it should be].

Check the graph in src/data/hospital.js: missing paths, paths that only work in one direction, a place on the wrong floor, wrong distances, a lift that doesn't connect every floor, and whether "Avoid stairs" removes stair paths before the route is worked out. Print the route as a list of places to find where it goes wrong, then fix the data or the code. Afterwards, test 10 pairs of places, including floor changes with "Avoid stairs" on and off, and show me the results as a table.

DEBUG · IT BROKE AFTER MY LAST CHANGE Copy

Everything worked until my last change. Now [what's broken].
The change I made, or asked you to make: [describe it or paste it]

Compare the code before and after that change and find what broke. Fix it so the new change and everything that worked before both work. If the change can't work without breaking something, tell me and suggest another way to do it.

DEBUG · THE AI KEEPS GOING IN CIRCLES Copy

Stop. We've tried to fix this several times and it still isn't fixed. Don't write any more code yet.
The problem: [describe it]
1. List what we've already tried, and why each attempt didn't work.
2. List three possible root causes, most likely first, and what would confirm each one.
3. Tell me exactly what to check or log (one console.log, or one thing to look at) to find out which it is.
Once I tell you the result, make one focused fix.

DEBUG · BROKEN ON A PHONE Copy

scrolls sideways, no text is cut off, buttons are at least 56px tall and don't overlap, long Hindi words wrap, and the floor plan scales to fit the screen. Check every screen, not only the one I named, and tell me which ones you changed.

DEBUG · BROKEN AFTER DEPLOYING Copy

The app works on my laptop but not after deploying to [Vercel / Netlify / GitHub Pages].
What happens: [e.g. a blank page, a 404 when I refresh /staff, the queue doesn't sync]
The build log or browser console shows: [paste it]

Check the usual causes one by one: the build command and output folder; the base path; a rewrite so every route serves index.html; file names whose capital letters differ from their imports (my laptop may ignore case, but the server doesn't); environment variables that exist on my laptop but were never added to the host; and, if we have a server, whether the app points at the server's live address instead of localhost. Tell me exactly which settings to change and where.

GUIDES FOR THIS PROBLEM

Ideation

Finding your idea

A two-hour method for turning a problem statement into one clear idea you can build...

Open guide

Playbook

Your working demo in 48 hours

A plan for the whole weekend, whether or not you code: pick a problem, build it with ...

Open guide

Resources

Tech resource library

The tools, data and building blocks worth using, what each plan actually allows, and...

Open guide

Template

Pitch deck template

A finished 7-slide sample deck you copy in one click and turn into your own pitch.

Open guide

HACK CACHE

Back

Nitin Raina · Co-Founder
nitin.cto@codenabled.in
+91 7051133258

Aryaman Tickoo · Co-Founder
aryaman.ceo@codenabled.in
+919682599189

Reeya Rehan · CMO
reeya.cmo@codenabled.in
+91 91497 39117

Dr. Simmi Dutta · HOD C.E. DEPTT GCET JAMMU
simmi.dutta@gcetj.ac.in

CachedInfo · Support Team
mail@cachedinfo.in

© 2026 Hackcache

Powered by cachedinfo